

Computer Security

Lecture 3 Notes

Memory Safety

Based on slides by Roberto Guanciale

Contents

1	Introduction	2
2	Buffer Overflows and Buffer Overreads	2
3	A Simple Buffer-Overread Example	2
4	Stack Placement of Local Variables	3
5	Fixing the Overread	3
6	Heartbleed	4
7	Buffer Overflow and Authentication Bypass	5
8	Finding and Exploiting Buffer Overflows	5
9	Memory Safety in Different Languages	6
10	Process Memory Layout	6
11	Stack Frames and Return Addresses	7
12	Stack Buffer Overflows and Return Attacks	7
13	Shellcode	7
14	The Original Xbox Example	8
15	Global and Heap Overflows	8
16	Dangling Pointers and Use After Free	9
17	Uninitialized Memory	9
18	Double Free and Memory Leaks	10
19	Code-Reuse Attacks	11
20	A ROP Example on ARMv7	11
21	Address Knowledge and Defenses	12

22 Joypad-Only Code Injection in Super Mario World	12
23 Exercise	12
24 Exercise	13

1 Introduction

Memory-safety bugs are one of the most important classes of security vulnerabilities in real systems. They occur when a program accesses memory in a way that does not respect the boundaries, lifetime, or intended type of the objects stored there. A single mistaken access may let an attacker read secrets, corrupt critical data, crash a service, or redirect control flow to code chosen by the attacker.

These bugs are especially common in low-level languages such as C, C++, and assembly. Those languages were designed with performance and control in mind: they expose memory addresses, allow manual allocation and deallocation, and usually avoid automatic runtime checks. That flexibility is useful for operating systems, embedded software, libraries, and performance-critical programs, but it also means that a small programming mistake can become a security vulnerability.

The problem is not restricted to poorly engineered software. Memory-safety failures have affected widely deployed systems and libraries. Examples include the original Xbox softmodding vulnerabilities, Heartbleed in OpenSSL, and Stagefright in Android media processing. These cases are useful reminders that memory safety is a practical engineering concern, not only a theoretical topic.

2 Buffer Overflows and Buffer Overreads

A **buffer** is a contiguous region of memory used to store data, such as a string, a packet, an image block, or an array of integers. A **buffer overflow** happens when a program writes beyond the end of that region. A **buffer overread** happens when a program reads beyond the end of that region.

The two bugs are closely related. In both cases, the program escapes the object it was supposed to access and reaches adjacent memory. The difference is the direction of the security consequence:

- an overflow can corrupt neighboring data and change future program behavior;
- an overread can disclose neighboring data that the program should not reveal.

The exact effect depends on what is stored next to the vulnerable buffer. The neighboring memory may contain ordinary variables, passwords, cryptographic keys, heap metadata, function pointers, frame pointers, return addresses, or other control data. This is why a bug that at first appears to be a simple off-by-one or missing length check can become a full security compromise.

3 A Simple Buffer-Overread Example

Consider the following simplified C program. It has two local buffers, one for a name and one for a password. The input name is copied into the first buffer, and then the program prints a user-controlled number of bytes from that buffer.

```

int main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int n;

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    for (int i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");
}

```

If the user asks the program to print more bytes than `name` contains, the program continues reading the bytes that follow `name` in memory. In the example from the slides, the password buffer is nearby, so printing too many bytes from `name` also prints bytes from `pwd`. The attacker does not need direct access to the process memory. The vulnerable program reads the memory on the attacker's behalf and sends it to the output.

This is a confidentiality violation. The attacker has not changed the password, but has learned information that should have remained secret.

4 Stack Placement of Local Variables

The buffers in the previous example are local variables. Local variables are normally stored on the **stack**. The stack is the part of a process address space used for function calls, local variables, arguments, saved frame information, and return addresses.

When the example prints the addresses of `name` and `pwd`, their addresses are close to each other. In the concrete run used in the slides, the distance is 16 bytes. The two arrays are not necessarily exactly adjacent because the compiler may insert padding or choose a particular alignment, but they are still located in the same stack frame and close enough that an out-of-bounds read from one can reach the other.

This is an important point: memory-safety bugs are not abstract. They become exploitable because real compilers place real objects at concrete addresses. An attacker often needs to understand this layout, or infer it by testing, before a bug can be turned into a useful exploit.

5 Fixing the Overread

The direct fix is to ensure that the program never reads more bytes than the buffer contains. For the example above, the code should reject or clamp values of `n` that are larger than the size of `name`.

```

if (n > sizeof(name)) {
    n = sizeof(name);
}

```

This prevents the loop from reading beyond the buffer. If the program is meant to handle strings, it should also be careful about the null terminator. A buffer of size 8 can store at most 7 ordinary characters plus `\0`.

6 Heartbleed

Heartbleed is a famous real-world buffer-overread vulnerability in OpenSSL. It affected the TLS heartbeat extension. A heartbeat message contains a claimed payload length and a payload. The receiver is supposed to send the payload back, which lets one endpoint check that the other endpoint is still alive.

The bug was that OpenSSL trusted the length field supplied by the peer. If the peer sent a small payload but claimed a much larger payload length, vulnerable OpenSSL versions copied and returned more bytes than the peer had actually provided. Those extra bytes came from adjacent process memory.

The vulnerable logic was essentially this:

```
hbtype = *p++;
n2s(p, payload);
p1 = p;

buffer = OPENSSL_malloc(1 + 2 + payload + padding);
bp = buffer;

*bp++ = TLS1_HB_RESPONSE;
s2n(payload, bp);
memcpy(bp, p1, payload);
bp += payload;
RAND_pseudo_bytes(bp, padding);

ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer,
                 3 + payload + padding);
```

The critical line is the `memcpy`. The value `payload` comes from the network message. The pointer `p1` points to the actual bytes supplied by the peer. The code allocates a response buffer large enough for the claimed length and then copies `payload` bytes from `p1`. What is missing is a check that the received heartbeat record actually contains that many bytes.

The patch added a bounds check before the response is constructed:

```
if (1 + 2 + payload + 16 > s->s3->rrec.length)
    return 0;
```

This check verifies that the heartbeat type byte, the two-byte payload length field, the claimed payload, and the minimum padding all fit within the received record. If they do not fit, the function returns without sending a response.

The consequence of Heartbleed was an information leak from the memory of the TLS process. Depending on what happened to be in memory, the leak could expose session cookies, user data, private keys, or other sensitive values. The same bug could be triggered repeatedly, each time sampling another piece of memory. The response did not require authentication, so internet-facing services were especially exposed.

Fixing Heartbleed therefore required more than installing a patched OpenSSL library. Operators also had to assume that secrets might already have leaked: certificates had to be replaced, private keys regenerated, old certificates revoked, and user sessions or passwords reconsidered depending on the service.

7 Buffer Overflow and Authentication Bypass

A write beyond the end of a buffer can be even more dangerous than an overread. The authentication example uses two local buffers:

```
char name[8];
char pwd[8];

strcpy(pwd, "pwd0");
strcpy(name, argv[1]);

if (strcmp(pwd, argv[2]) == 0) {
    printf("authorized\n");
} else {
    printf("not authorized\n");
}
```

The intended behavior is simple. The program copies the real password into `pwd`, copies the supplied user name into `name`, and then checks whether the supplied password equals `pwd`. If the supplied password is `pwd0`, access is granted. Otherwise, access is denied.

The vulnerability is the unbounded copy into `name`. If the attacker supplies a name longer than the space available in `name`, the extra bytes continue into neighboring stack memory. In the concrete layout used by the example, `pwd` is 16 bytes away from `name`. A name such as

1234567890123456hello

fills the first 16 bytes and then overwrites `pwd` with `hello`. The attacker can now supply `hello` as the second argument. The comparison succeeds, even though the original password was not known.

This is an integrity violation. The attacker does not learn the original password. Instead, the attacker changes the program state so that the password check compares against attacker-controlled data.

String termination matters here. C string functions stop at the first null terminator. If the attacker overwrites only the beginning of `pwd` and the copied string includes a null terminator, `strcmp` may compare only the overwritten prefix. Different comparison logic, for example fixed-size comparison of cryptographic keys, would change the exact exploit requirements.

8 Finding and Exploiting Buffer Overflows

To exploit a buffer overflow, an attacker needs more than the knowledge that an overflow exists. The attacker must identify:

- which input reaches the vulnerable buffer;
- how the buffer is placed in memory;
- what data is adjacent to the buffer;
- how many bytes are needed to reach the target data;
- what effect the corrupted data has on later execution.

There are several ways to gather this information. If source code is available, the attacker can inspect it directly. If not, the attacker can analyze the binary with tools such as a disassembler or debugger. Fuzzing can also help: the attacker repeatedly runs the program with many generated inputs and watches for crashes or unexpected behavior.

Old vulnerabilities are also useful to attackers. Public patches often reveal where a bug was fixed. An attacker can inspect the patch, identify the affected versions, and then search for systems still running an unpatched release.

9 Memory Safety in Different Languages

At the assembly level, memory is just a large array of bytes. The processor does not know whether a particular byte sequence represents a string, an integer, a pointer, a stack frame, or code. The meaning comes from the instructions that operate on those bytes. Stack management, calling conventions, and object layout are explicit responsibilities of the compiler, runtime, or programmer.

C and C++ provide more structure. Programmers can define arrays, structs, classes, and functions. The compiler automatically manages stack frames when functions are called and returned from. However, the languages still allow raw pointers, pointer arithmetic, casts between pointer types, manual heap allocation, and manual deallocation. A programmer can cast an integer to a pointer and try to access that address. The language does not generally enforce that the access is valid.

Higher-level languages such as Java, Python, and ML enforce a stronger notion of type and object validity. Objects carry type information, array accesses are checked, invalid casts fail, and memory is reclaimed by a garbage collector. This removes many direct buffer-overflow and use-after-free bugs.

The tradeoff is overhead and, in some domains, unpredictability. Garbage collection can introduce pauses. For a web application, that may be acceptable. For a real-time controller, such as software involved in braking, unpredictable execution time may be a serious problem. This is one reason why low-level languages remain common in operating systems, embedded systems, libraries, and legacy infrastructure.

10 Process Memory Layout

A process normally has a private virtual address space divided into regions. Although exact addresses vary across operating systems and configurations, the basic layout is usually:

- code and libraries at lower addresses;
- global data above the code;
- the heap above global data;
- the stack near high addresses.

The code region contains the machine instructions of the program and its loaded libraries. The global-data region contains variables that exist for the whole lifetime of the process. The heap contains dynamically allocated objects, created with functions such as `malloc` or operators such as `new`. The stack contains function-call frames and is managed automatically as functions are called and return.

The heap and stack serve different purposes. Heap objects live until explicitly freed, or until a garbage collector proves that they are no longer reachable. Stack objects live only while their function invocation is active. When the function returns, its stack frame is removed.

Modern systems use virtual memory, so the addresses seen by a process are not raw physical addresses. Each process can appear to have its own private memory, even if different processes use the same virtual address values for different physical memory locations.

11 Stack Frames and Return Addresses

Each function call creates a stack frame. The frame contains local variables, saved registers, arguments or argument spill slots, the saved frame pointer, and the return address. The exact layout depends on the architecture, compiler, optimization level, and calling convention, but the principle is the same: when a function returns, the processor must know where execution should resume.

Suppose `main` has a local buffer and calls a function `hello`. The new function invocation gets a new frame below the previous one because, on many systems, the stack grows from high addresses toward lower addresses. If `main` passes the address of its local buffer to `hello`, then the argument in the new frame is a pointer back into the older frame.

The return address cannot simply be hard-coded into the called function. The same function may be called from many places. Therefore, each specific function call saves its own return address on the stack. When the function finishes, the saved return address is loaded back into the program counter, and execution continues after the call site.

This mechanism is essential for normal execution. It also creates a security target: if an attacker can overwrite the saved return address, the attacker can influence where the program jumps next.

12 Stack Buffer Overflows and Return Attacks

A stack buffer overflow occurs when data written into a local stack buffer continues past the buffer and reaches the rest of the frame. At first, the overflow may corrupt ordinary local variables. With enough bytes, it can also corrupt the saved frame pointer and the saved return address.

If the return address is replaced with an invalid address, the program will usually crash. That is already a denial of service. If the return address is replaced with the address of useful existing code, the program may execute that code instead of returning normally.

For example, if the attacker knows the address of a function `hello`, the attacker can place that address where the saved return address is stored. When the current function returns, control jumps to `hello`. In the example from the slides, this can even make the function call itself repeatedly.

To make such a control-flow subversion work, the attacker needs several pieces of information:

- the address of the target code;
- the number of bytes from the vulnerable buffer to the saved return address;
- a value for the saved frame pointer that does not crash the program too early;
- the byte order of the machine, such as little endian or big endian.

13 Shellcode

Instead of redirecting execution to an existing function, an attacker may try to redirect execution to bytes supplied as part of the input. Those bytes are called **shellcode**. The name comes from the classic payload of spawning a shell, but shellcode can do many other things: open a network connection, modify files, download additional code, or invoke system calls directly.

A shellcode attack typically has two parts:

- place machine code in memory, often inside the overflowed buffer;
- overwrite a return address so that execution jumps to that buffer.

Shellcode must satisfy strict constraints. It is architecture-specific because it consists of machine instructions. It is usually operating-system-specific because system calls and calling conventions differ. It must also avoid bytes that would terminate the input early. For example, if the vulnerable program uses `gets`, a newline byte may stop the copy. If the input is processed as a C string, null bytes may be forbidden because they terminate the string.

A typical exploit layout is therefore:

```
[ shellcode ][ padding ][ saved frame pointer ][ return address ]
```

The return address is chosen to point back into the buffer where the shellcode was placed. When the vulnerable function returns, the processor starts executing attacker-supplied bytes as instructions.

14 The Original Xbox Example

The original Xbox is a useful example of exploit chaining. Microsoft used a dashboard and code-signing checks to restrict which software could run on the console. Attackers found buffer-overflow vulnerabilities that could be reached through game data and dashboard components.

The chain described in the recording used one overflow to bootstrap a small payload, such as an FTP server. That payload allowed additional files to be loaded onto the console. A second vulnerable component, such as font loading in the dashboard, then processed attacker-controlled data and triggered another overflow. The second stage gave the attacker the ability to execute unsigned code.

This kind of chain is common in real exploitation. The first bug may not be powerful enough to achieve the final goal directly. Instead, it creates a foothold that enables a second step, which then enables a stronger payload.

15 Global and Heap Overflows

Not all overflows happen on the stack. Programs also store data in global memory and on the heap.

A global-data overflow corrupts objects that live for the whole lifetime of the program. It usually cannot overwrite a stack return address directly because the return address is not stored in the global-data region. However, global memory may contain sensitive values, configuration flags, function pointers, or objects used later by the program.

A heap overflow corrupts dynamically allocated objects. Heap overflows also do not directly overwrite a stack return address. They may still be exploitable because heap objects often contain pointers, object metadata, virtual-method tables, or application-level control data. In C++, for example, object dispatch may rely on function pointers. If an attacker corrupts such a pointer, the next method call may redirect control flow.

Heap exploitation is often more complex than the small stack examples because the attacker must understand allocator behavior and heap layout. Still, the underlying security problem is the same: an input intended for one object modifies memory belonging to another object.

16 Dangling Pointers and Use After Free

A **dangling pointer** is a pointer that does not refer to a valid object of the expected type anymore. One source is an incorrect cast. For example, a program may store both `Dog` and `Cat` objects in a generic list, retrieve a pointer, and cast it to the wrong type. If `Cat` has more fields than `Dog`, reading a `Cat` field through a pointer to a `Dog` object reads beyond the actual object. Writing that field corrupts the memory after the object.

Another source is use after free:

```
Dog *dog = malloc(sizeof(Dog));
free(dog);

Cat *cat = malloc(sizeof(Cat));
dog->x = 1;      /* dog may now refer to memory reused for cat */
```

After `free(dog)`, the allocator is allowed to reuse that memory for a new object. If the program continues using `dog`, it may accidentally read or write the new object. In a garbage-collected language, this specific mistake is harder to make because the runtime does not reclaim an object while there are still references to it.

Stack lifetime can create another dangling-pointer bug. A function should not return a pointer to one of its local variables:

```
Dog *f(void) {
    Dog dog;
    return &dog;      /* invalid after f returns */
}
```

The object `dog` lives in the stack frame of `f`. When `f` returns, that frame is removed. The pointer still contains an address, but the object at that address is no longer valid.

17 Uninitialized Memory

Uninitialized memory is another memory-safety problem. A local variable may reuse stack memory that previously belonged to another function call. Compilers do not generally wipe stack frames when functions return. They only adjust the stack pointer. Therefore, old bytes may remain until overwritten.

The first example is a correctness and memory-corruption problem. Suppose a function initializes a local integer, returns, and then another function later declares a local pointer without initializing it.

```
void f(void) {
    int a = 42;
}

void g(void) {
    Dog *x;
    woof(x);
}
```

When `f` returns, its stack frame is removed logically, but the bytes are not necessarily erased. If `g` reuses the same stack location, `x` may contain the old bit pattern left by `a`. The program then uses that stale value as if it were a valid pointer. In the example, the value 42

may be interpreted as an address, even though no `Dog` object was allocated there. The result is undefined behavior: the program may crash, read from an unintended location, or write to memory that belongs to something else.

The second example is a confidentiality problem. A function may store a secret in a local stack buffer and return without clearing it:

```
void f(void) {
    char pwd[8] = "123456";
}

void g(void) {
    char public[8];
    printf("%s\n", public);
}
```

The buffer `pwd` exists only while `f` is active, but its bytes may remain on the stack after `f` returns. If `g` later receives a stack frame that overlaps the same memory, the uninitialized array `public` may contain the old password bytes. Printing `public` can therefore leak `123456`, even though `g` never explicitly received the password.

The lesson is simple: programs should initialize memory before reading it, and should explicitly clear sensitive data when keeping it in memory longer than necessary would be dangerous.

18 Double Free and Memory Leaks

A **double free** occurs when the same heap object is released twice.

```
Dog *dog = malloc(sizeof(Dog));
free(dog);
free(dog);          /* incorrect */
```

Allocators keep internal metadata describing which chunks of memory are free and which are in use. Freeing the same chunk twice can corrupt that metadata or cause later allocations to return the same memory to two different parts of the program. Modern allocators often include checks for this, but relying on the allocator to catch the bug is not a safe programming practice.

A **memory leak** is the opposite lifetime problem. The program allocates memory and then loses the ability to free it.

```
void f(void) {
    malloc(64);
}

for (;;) {
    f();
}
```

Each call allocates 64 bytes and never releases them. Over time, the process consumes more and more memory. Eventually, the program or the whole service may run out of memory, causing a denial of service.

19 Code-Reuse Attacks

Modern systems often include defenses that make direct code injection harder. For example, the stack may be marked non-executable, so bytes placed in an input buffer cannot simply be executed as instructions. Code-reuse attacks avoid this defense by not injecting new code. Instead, they reuse instructions already present in the program or its libraries.

The most important form is **return-oriented programming**, or ROP. A ROP attack chains together short instruction sequences called **gadgets**. Each gadget performs a small operation and then transfers control indirectly, often through a return instruction. By arranging values on the stack, the attacker can choose the next gadget and pass data between gadgets.

The goal is not usually to implement a large program efficiently. The goal is often to prepare arguments and then jump to an existing function or system call, such as one that executes a command, opens a file, changes memory permissions, or sends data over the network.

20 A ROP Example on ARMv7

The slides use a small ARMv7 example because the gadget behavior is easier to follow than in many x86 examples. Suppose the attacker wants to copy the value stored at address X into address Y. In C-like notation, the goal is:

```
*Y = *X;
```

Assume the program contains the following gadgets:

```
g1: pop {r0, r4, pc}
g2: ldr r0, [r0]
    pop {r4, pc}
g3: str r0, [r4, #0x10]
    pop {r4, pc}
```

The attacker first corrupts a return address so that execution returns to **g1**. The stack is arranged so that each **pop** loads registers and also chooses the next gadget by loading **pc**, the program counter.

```
return address: g1
next stack word: X
next stack word: 0
next stack word: g2
next stack word: Y - 0x10
next stack word: g3
next stack word: 0
next stack word: next
```

Execution proceeds as follows:

- g1 pops X into r0, pops 0 into r4, and pops g2 into pc;
- g2 executes `ldr r0, [r0]`, so r0 becomes the value stored at address X;
- g2 then pops Y - 0x10 into r4 and g3 into pc;
- g3 executes `str r0, [r4, #0x10]`, which stores the value from r0 at address Y;
- the final pop moves execution to the next attacker-selected location.

The attacker has computed $*Y = *X$ without injecting code that directly implements that assignment. The computation was assembled from short fragments already present in the program.

Real ROP attacks often rely on large libraries such as `libc` because they contain many possible gadgets. In principle, enough gadgets can provide a Turing-complete computation model. In practice, attackers usually chain just enough gadgets to make one powerful call.

21 Address Knowledge and Defenses

The examples above assume that the attacker knows useful addresses: the address of a target function, the address of a buffer, or the addresses of ROP gadgets. This assumption is reasonable in a basic setting where the program layout is fixed and the attacker can inspect the binary.

Modern systems try to make this harder with defenses such as address-space layout randomization. ASLR shifts code, libraries, heap, and stack locations by random offsets when the program starts. If the attacker does not know the actual runtime address of a gadget or buffer, a hard-coded exploit is likely to crash.

Attackers therefore often chain vulnerabilities. A first memory-disclosure bug may leak a pointer. Once one pointer is known, the attacker can infer nearby addresses because the relative distances inside a binary or library are often fixed. A second bug can then use the recovered address information to perform a control-flow attack.

22 Joypad-Only Code Injection in Super Mario World

A striking high-level example of these ideas is the code-injection attack against *Super Mario World* that can be performed using only ordinary controller input. No cartridge modification, external debugger, or hardware implant is needed. The attacker manipulates the game state through a carefully chosen sequence of joypad actions.

The attack is interesting because the controller is not normally thought of as a code-loading interface. It is only supposed to move the character, jump, run, select items, and interact with the game world. However, game state is still data in memory. If the game contains memory-safety bugs or unexpected state transitions, controller input can be used to drive that memory into abnormal configurations.

At a high level, the attack chains several effects together. First, the player uses precise inputs to trigger a sequence of memory corruptions. One important step is to corrupt Mario's power-up state so that the game believes Mario is in a state that should not normally exist. The game contains code paths and function-pointer-like dispatch behavior for handling different states. When an invalid state is reached, the game can fetch control information from an unexpected memory location.

That control-flow mistake is then used to make the game interpret parts of controller-controlled state as data for a write. The player can influence values such as Mario's position, movement, and related game variables. With enough precision, those values can be made to represent bytes of machine code or addresses. Repeating the process lets the attacker write a small program into memory one piece at a time.

The final result is arbitrary code execution produced entirely through normal joypad input.

23 Exercise

Exercise. Consider the following C program:

```
#include <stdio.h>
#include <string.h>
```

```

int main(int argc, char **argv) {
    char name[8];
    int authorized = 0;

    strcpy(name, argv[1]);

    if (authorized) {
        printf("access granted\n");
    } else {
        printf("access denied\n");
    }
}

```

Assume that, in one concrete execution, the compiler places `authorized` immediately after `name` in memory. Explain why this program is vulnerable. Give one input strategy that could make the program print `access granted`, and explain why simply changing the value of `authorized` is different from learning a secret password.

Solution

The vulnerability is the call to `strcpy`. The destination buffer `name` has space for only eight bytes, but `strcpy` copies until it reaches the null terminator in the attacker-controlled input. If the input is longer than the space available in `name`, the extra bytes are written past the end of the buffer.

Under the assumption in the question, the memory after `name` contains `authorized`. Therefore, a sufficiently long argument can overflow `name` and overwrite `authorized` with a nonzero value. Since C treats any nonzero integer as true in an `if` condition, the program can then execute the `access granted` branch.

This is an integrity violation rather than a confidentiality violation. The attacker does not learn a password or read a secret value. Instead, the attacker changes the program state so that the authorization check uses a corrupted value.

Wrong solution

The program is safe because `name` has eight bytes and the input is a string, so C will stop copying when the buffer is full. Therefore, the only way to print `access granted` is to know the correct password or to modify the source code.

This reasoning is wrong because `strcpy` does not know the size of the destination buffer. It stops at the null terminator in the source string, not at the end of `name`. If the source string is too long, the copy continues into adjacent memory.

24 Exercise

Exercise. Consider the following C-like program:

```

typedef struct {
    int is_admin;
    char name[16];
} User;

typedef struct {

```

```

    int amount;
    char label[16];
} Payment;

int main(void) {
    User *u = malloc(sizeof(User));
    u->is_admin = 0;

    free(u);

    Payment *p = malloc(sizeof(Payment));
    p->amount = 1;

    if (u->is_admin) {
        printf("admin action allowed\n");
    }
}

```

Is the program memory safe? If not explain the bug and what can be the consequence for the program.

Solution

The bug is a use-after-free. The pointer `u` is still used after the object it pointed to has been released with `free(u)`. Once the memory is freed, the allocator is allowed to reuse it for another allocation.

`p` may be allocated at the same heap address that `u` used to occupy. That means `u` and `p` now refer to the same bytes, but the program interprets those bytes using two different types. The assignment `p->amount = 1` writes the value 1 into the first integer field of the `Payment`. Since `u->is_admin` also reads the first integer field at the same address, the old pointer `u` sees that value as an administrator flag.

The program can therefore print `admin action allowed` because stale pointer `u` aliases a new object. The attacker does not need to write past the end of a buffer. The problem is that the program continues to trust a pointer after the lifetime of the original object has ended.

Wrong solution

The bug is a buffer overflow because `Payment` overwrites `User`. The fix is to make `name` and `label` larger so that the two structures do not overlap.

This reasoning is wrong because no copy writes beyond the end of either structure. The overlap happens because the heap allocator legitimately reuses memory after `free`. Making the arrays larger does not fix the lifetime bug. The correct fix is to stop using `u` after it has been freed, for example by setting it to `NULL` and ensuring all later code obtains a valid live object before dereferencing it.